

Sequential Logic & Memory

Lab 3 — Nand2Tetris Project 3

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 21, 2026

Aim. *To move from combinational to clocked sequential design: building a 1-bit register from the primitive DFF gate, and composing it upward into byte-, word- and block-addressable RAM arrays, plus the Program Counter.*

3.1 Learning Outcomes

On completing this lab, a student will be able to:

- Distinguish combinational circuits (no state) from sequential circuits (state + clock).
- Explain how a D flip-flop (DFF) stores one bit across a clock cycle.
- Build multi-bit registers and hierarchically-addressed RAM ($\text{RAM8} \rightarrow \text{RAM64} \rightarrow \text{RAM512} \rightarrow \text{RAM4K} \rightarrow \text{RAM16K}$) using the same recursive pattern at every level.
- Build a Program Counter (PC) supporting load, increment and reset, as used in instruction fetch.

3.2 Pre-Lab Reading

Pre-Lab Reading. The DFF itself is prerequisite material: this course’s stated prerequisite, Digital Logic Design, already covers the D flip-flop’s clocked storage behaviour (state holds between clock edges; a new value is latched only on the active edge) — this lab treats DFF as the supplied sequential primitive and does *not* re-derive its internal transistor/gate-level design.

What CS401 itself contributes here is register-level organization: **Week 4** introduced the CPU’s program-visible registers as named, addressable storage elements on a common bus (Mano, *Computer System Architecture*, 3rd ed., Sec. 8-2, p.242, and the common-bus system of Sec. 5-2, p.127-129) — this lab builds that same idea (a register: load a value, hold it, read it back) bottom-up from a single DFF rather than top-down from an architectural block diagram. Chapter-level only for Hamacher Ch. 7 (register/operand treatment), per the textbook-grounding rule.

New in this lab, not covered by any CS401 lecture: the Nand2Tetris *recursive RAM pattern* — $\text{RAM}(n)$ is built from 8 $\text{RAM}(n/8)$ units plus a $\text{DMux8Way}/\text{Mux8Way16}$ pair (Nisan & Schocken, *The Elements of Computing Systems*, Ch. 3, *Sequential Logic*) — read that chapter before attempting RAM64 and above.

3.3 Toolchain Setup

1. Install a Java Runtime Environment (JRE), version 8 or later, if not already present.
2. Download the Nand2Tetris software suite (free, nand2tetris.org) — this lab uses the **Hardware Simulator** only.
3. Extract the suite and locate the `projects/03/` folder — it contains `a/` (Bit, Register, RAM8, RAM64) and `b/` (RAM512, RAM4K, RAM16K, PC) subfolders, each pre-populated with a `ChipName.hdl` stub and the supplied `ChipName.tst` / `ChipName.cmp` test/comparison scripts. These scripts are standard Nand2Tetris project files and are *not* redistributed here — obtain them from the official suite.
4. Open the Hardware Simulator, load a chip's `.tst` script (FILE → LOAD SCRIPT), and run it (SIMULATION → RUN). A correct chip prints `End of script --- comparison ended successfully` in the script's output pane; a wrong one reports the first mismatching row. Sequential chips only update their output *after* a clock pulse (TICK, then TOCK) — the `.tst` script drives this for you.
5. RAM4K and RAM16K simulations run visibly slower than the combinational chips from Labs 1-2 — this is expected (thousands of internal DFFs), not a sign of a broken chip.

3.4 Tasks

Implement the following 7 chips **in order** — `Bit` → `Register` → `RAM8` establishes the recursive pattern; `RAM64` through `RAM16K` repeat it at increasing scale; `PC` is independent (built from `Register`, `Inc16`, and `Mux16` from Labs 1-2).

1. `Bit(in,load) → out` — a 1-bit register. On a clock edge, if `load` is asserted, `out` takes the value of `in`; otherwise `out` holds its previous value. Built from one DFF and one Mux feeding the DFF's input from either `in` (if loading) or the DFF's own current `out` (feedback, if holding).
2. `Register(in[16],load) → out[16]` — sixteen `Bit` chips in parallel, sharing one `load` line.
3. `RAM8(in[16],load,address[3]) → out[16]` — 8 registers, address-selected. A `DMux8Way` routes `load` to exactly one of 8 `Registers` (selected by `address`); a `Mux8Way16` selects that same register's output onto `out` every cycle (read is combinational; write is clocked).
4. `RAM64(in[16],load,address[6]) → out[16]` — 8 `RAM8` units, address-selected identically: the top 3 bits of `address` pick a `RAM8`, the bottom 3 bits pass through to it unchanged.
5. `RAM512`, `RAM4K`, `RAM16K` — the same recursive pattern, one level deeper each time ($8 \times \text{RAM64}$, $8 \times \text{RAM512}$, $4 \times \text{RAM4K}$ respectively — `RAM16K` uses `DMux4Way/Mux4Way16` instead of the 8-way variants, since 16K needs only a 2-bit top address split across 4, not 8, children).
6. `PC(in[16],load,inc,reset) → out[16]` — the Program Counter, built from one `Register` plus an `Inc16` and `Mux16` chain. On each clock edge: if `reset`, load 0; else if `load`, load `in`; else if `inc`, load `out+1`; else hold. **Priority order matters:** `reset` > `load` > `inc` > hold.

3.5 Figures

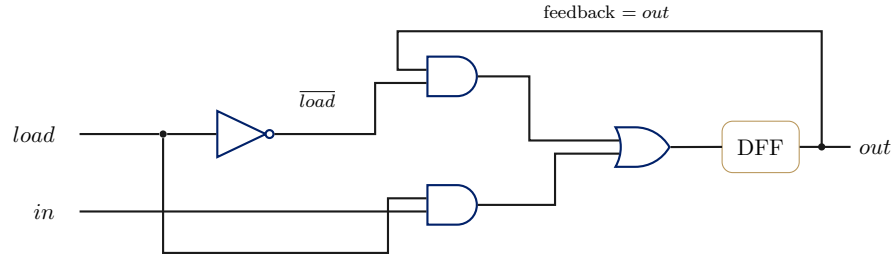


Figure 1: **Bit** at gate level: the Mux is $out_{next} = (load \cdot in) + (\overline{load} \cdot out)$ — the top **And** holds the current value, the bottom **And** captures a new value when **load** is asserted, and the DFF latches the result on the clock edge.

load	in	out (before)	out (after)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Figure 2: **Bit** behaviour table (after a clock edge).

Step-by-step Boolean expression of **Bit**:

Mux select: $d_{in} = load ? in : out$

expand Mux: $d_{in} = (\overline{load} \cdot out) + (load \cdot in)$

DFF latch: $out_{next} = d_{in} = \overline{load} \cdot out + load \cdot in$

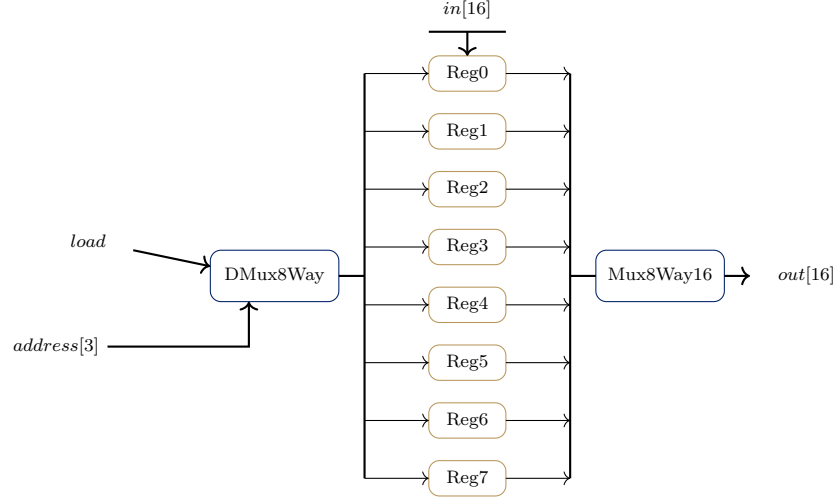


Figure 3: **RAM8**: **DMux8Way** routes the write pulse to exactly one **Register**’s load line via a vertical fan-out bus; **Mux8Way16** collects every register’s output on a second bus and continuously selects one onto **out**. **in[16]** is wired identically into every register (only **Reg0**’s tap is drawn, to keep the diagram legible). **RAM64** and above repeat this exact wiring pattern with the child being another **RAM(n/8)** instead of a bare **Register**.

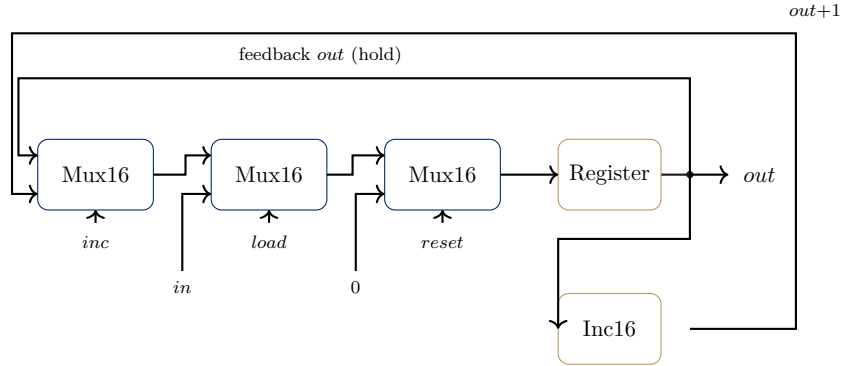


Figure 4: **PC**: a three-stage **Mux16** cascade enforces the priority **reset** > **load** > **inc** > **hold**. The right-most mux (**reset**) can override everything; the left-most (**inc**) only wins if neither **reset** nor **load** is asserted. **Inc16** and the hold path both read the register’s own **out**.

reset	load	inc	out_{next}
1	–	–	0
0	1	–	in
0	0	1	$out + 1$
0	0	0	out

Figure 5: **PC** priority table (“–” = don’t care).

Step-by-step **PC** Boolean expression:

$$\begin{aligned}
 s_{inc} &= inc ? (out+1) : out \\
 s_{load} &= load ? in : s_{inc} \\
 out_{next} &= reset ? 0 : s_{load}
 \end{aligned}$$

i.e. **reset** checked first, then **load**, then **inc**, else hold.

3.6 Observation / Journal Table

Record, for each chip, its internal composition and whether it passed its supplied test script.

Chip	Internal composition	Pass / Fail
Bit		
Register		
RAM8		
RAM64		
RAM512		
RAM4K		
RAM16K		
PC		

Table 1: Lab 3 observation table — one row per chip.

Journal instruction (separate from the table above): draw the block diagram showing how RAM64 is composed from $8 \times \text{RAM8}$ (Figure 3 shows the pattern one level down — reproduce it one level up, in your own hand, for the journal). This is the syllabus’s own Deliverable 2.

3.7 Viva-Voce Questions

1. What distinguishes a combinational circuit (Labs 1-2) from a sequential one (this lab)? Where does "state" physically live in Bit?
2. Trace Bit by hand: if out currently holds 1 and load=0, what value does the Mux feed to the DFF’s input, and why does out stay 1 after the next clock edge?
3. Explain why RAM8’s load line needs a DMux8Way but its out line needs a Mux8Way16 instead — why not the same chip for both directions?
4. Explain the reset > load > inc priority order in PC. What would go wrong at instruction fetch if inc could override load?
5. Why does RAM16K use DMux4Way/Mux4Way16 instead of the 8-way chips used everywhere else in the recursion?

3.8 Rubric

Reproduced verbatim from syllabi/CS401_Lab_Syllabus_Nand2Tetris.pdf.

Criterion	Marks
All chips pass their test scripts, including RAM16K and PC	5
Correct load/store timing (value updates only on next clock edge)	2
Journal: recursive RAM block diagram correct	2
Viva: can explain reset > load > inc priority in PC	1
Total	10